Roll.No: A074	**Name:** Aakarsh Singh
Prog/Yr/Sem: B.Tech IT/2/3	**Batch:** A2
Date of Experiment: 2/9/2026	**Date of Submission:**

Q1.

i. Implement a program that takes a postfix expression as input and evaluates it using a stack to return the result.

Program

```
#include <stdio.h>
#include <ctype.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int value)
{
   stack[++top] = value;
}

int pop()
{
   return stack[top--];
```

```
}

int calculate(int a, int b, char op)
{
   switch (op)
   {
      case '+': return a + b;
      case '-': return a - b;
      case '*': return a * b;
      case '/': return a / b;
      case '%': return a % b;
   }
   return 0;
}

int evaluatePostfix(char exp[])
{
   int i, a, b;

   for (i = 0; exp[i] != '\0'; i++)
   {
      if (isdigit(exp[i]))
      {
         push(exp[i] - '0');
```

```
        }
        else
        {
            b = pop();
            a = pop();
            push(calculate(a, b, exp[i]));
        }
    }

    return pop();
}

int main()
{
    char postfix[MAX];

    printf("Enter postfix expression: ");
    scanf("%s", postfix);

    printf("Result = %d\n", evaluatePostfix(postfix));

    return 0;
}
```

Output

```
Enter postfix expression: 23*54*+9-
Result = 17
```

Q2.

ii. Implement a program that takes an infix expression as input and converts it into a postfix expression using a stack.

Program

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char ch)
{
   stack[++top] = ch;
}

char pop()
{
```

```
    return stack[top--];
}

int precedence(char op)
{
    if (op == '^')
        return 3;
    else if (op == '*' || op == '/' || op == '%')
        return 2;
    else if (op == '+' || op == '-')
        return 1;

    return 0;
}

void infixToPostfix(char infix[], char postfix[])
{
    int i, j = 0;
    char ch;

    for (i = 0; infix[i] != '\0'; i++)
    {
        ch = infix[i];
```

```
if (isalnum(ch))
{
   postfix[j++] = ch;
}

else if (ch == '(')
{
   push(ch);
}

else if (ch == ')')
{
   while (top != -1 && stack[top] != '(')
   {
      postfix[j++] = pop();
   }
   pop();
}

else
{
   while (top != -1 &&
         stack[top] != '(' &&
         precedence(stack[top]) >= precedence(ch))
```

```
            {
                postfix[j++] = pop();
            }

            push(ch);
        }
    }

    while (top != -1)
    {
        postfix[j++] = pop();
    }

    postfix[j] = '\0';
}

int main()
{
    char infix[MAX], postfix[MAX];

    printf("Enter infix expression: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);
```

```
    printf("Postfix expression = %s\n", postfix);

    return 0;
}
```

Output

```
Enter infix expression: A+B*C
Postfix expression = ABC*+
```

Q3.

iii. For each operation, identify the Push and Pop operations performed by the stack and analyze the time and auxiliary space complexity.

Program

```
#include <stdio.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int value)
{
    if (top == MAX - 1)
```

```
    {
        printf("Stack Overflow\n");
    }
    else
    {
        stack[++top] = value;
        printf("%d pushed into stack\n", value);
    }
}

void pop()
{
    if (top == -1)
    {
        printf("Stack Underflow\n");
    }
    else
    {
        printf("%d popped from stack\n", stack[top]);
        top--;
    }
}

void display()
```

```
{
    int i;

    if (top == -1)
    {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack elements: ");

    for (i = top; i >= 0; i--)
    {
        printf("%d ", stack[i]);
    }

    printf("\n");
}

int main()
{
    int choice, value;

    while (1)
```

```
{
    printf("\n--- STACK MENU ---\n");
    printf("1. Push\n");
    printf("2. Pop\n");
    printf("3. Display\n");
    printf("4. Exit\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            printf("Enter value: ");
            scanf("%d", &value);
            push(value);
            break;

        case 2:
            pop();
            break;

        case 3:
            display();
```

```
            break;

        case 4:
            return 0;

        default:
            printf("Invalid choice\n");
      }
   }
}
```

Output

```
--- STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value: 5
5 pushed into stack

--- STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack elements: 5
```

DOMS | Page No.
Date / /

DSA Lab 5

9. Operator Precedence and Associativity Table

Operator	Precedence	Associativity
( )	Highest	—
^	3	Right to Left
*, /	2	Left to Right
+, -	1	Left to Right

Precedence decides which operator binds tighter when two different operators compete for the same operand.
Associativity decides the order of evaluation when operators of the same precedence appear consecutively.

8. Postfix Evaluation using Stack

Step 1: Create empty Stack S
Step 2: for each token in expression:
    Step 3: If token is an operand:
    Step 4: push (S, token)

Step 5: else if token is an operator (+, -, *, /):
    Step 6: op2 = pop(s)
    Step 7: op1 = pop(s)
    Step 8: result = apply_operator (op1, token, op2)
    Step 9: push (S, result)

Step 10: return pop(s).

Time complexity: O(n).
Space complexity: O(n).

DOMS | Page No.
Date / /

Infix to Postfix conversion using Stack

Step 1 : Create empty stack S
Step 2 : Create empty string result

Step 3: for each token in expression:
  Step 4: if token is an operand:
  Step 5: result = result + token

Step 6: else if token == '(':
Step 7: push (S, token)

Step 8: else if token == ')':
Step 9: while top(S) != '(':
Step 10: result = result + pop(S)
Step 11: pop(S)

Step 12: else if token is an operator:
Step 13: while (S is not empty) AND (top(S) != '(') AND
(precedence(top(S)) > precedence(token) OR
(precedence(top(S)) == precedence(token) AND
token is left-associative)
Step 14: result = result + pop(S)
Step 15: push (S, token)

Step 16: while S is not empty:
Step 17: result = result + pop(S)

Step 18: return result

Time & Space complexity : O(n).

DOMS | Page No.
Date / /

Q. Infix to Postfix conversion using Stack

Step 1: reverse the infix expression
Step 2: While reversing, swap every '(' with ')' and every ')' with '('.

Step 3: Create empty stack S
Step 4: Create empty string result

Step 5: for each token in the reversed expression.
Step 6: if token is an operand
Step 7: result = result + token

Step 8: else if token == '(':
Step 9: push (S, token)
Step 10: else if token == ')':
Step 11: while top(S) != '(':
Step 12: result = result + pop(S)
Step 13: pop(S)

Step 14: else if token is an operator:
Step 15: while (S is not empty) AND (top(S) != '(') AND (precedence (top(S)) > precedence (token) OR (precedence(top(S)) == precedence (token) AND token is right-associative)):
Step 16: result = result + pop(S)
Step 17: push (S, token)
Step 18: while S is not empty
Step 19: result = result + pop(S)
Step 20: reverse the result string
Step 21: return result

Q. " a / b - c + d * e - a * c "

Expression	Stack	output
a	empty	a
/	/	a
b	/	ab
-	-	ab/
c	-	ab/c
+	+	ab/c+
d	+	ab/c+d
*	+*	ab/c+d
e	+*	ab/c+de
-	-	ab/c+de*+
a	-	ab/c+de*+a
*	-*	ab/c+de*+a
c	-*	ab/c+de*+ac

Infix to Postfix → Final Expression: ab/c+de*+ac*-

Infix to Prefix → " c * a - e * d + c - b / a "

Expression	Stack	output
c	empty	c
*	*	c
a	*	ca
-	-	ca*
e	-	ca*e
*	-*	ca*e
d	-*	ca*ed
+	+	ca*ed*-
c	+	ca*ed*-c
-	-	ca*ed*-c+
b	-/	ca*ed*-c+b
a	-/	ca*ed*-c+ba/-

Reverse ← ca*ed*-c+ba/-

DOMS | Page No.
Date / /

Q. Evaluate postfix evaluation: "546 + * 493 / + *"

Expression	op1	op2	output	Stack
5				5
4				5,4
6				5,4,6
+	4	6	10	5,10
*	5	10	50	50
4				50,4
9				50,4,9
3				50,4,9,3
/	9	3	3	50,4,3
+	4	3	7	50,7
*	50	7	350	350

∴ 546 + * 493 / + * = 350 ⇒ Ans

Q. Evaluate prefix expression: "/+33--+47*+123"

3 2 1 + * 7 4 + - • 3 3 + /

Expression	op1	op2	output	Exp
3				3
2				3,2
1				3,2,1
+	1	2	3	3,3
*	3	3	9	9
7				9,7
4				9,7,4
+	4	7	11	9, 11
-	11	9	2	2
•				
3				2,3
3				2,3,3
+	3	3	6	2,6
/	6	2	3	3

∴ Final Evaluation = 3 ⇒ Ans